

Smart Hospital LLM Project — End-to-End Overview

1. What This Project Is

This project builds a system that monitors patient vital signs (heart rate, oxygen level, breathing rate) and uses a local AI model to help hospital staff understand what's happening — in plain language — when something looks concerning.

It does **not** replace clinical judgment. Its role is to notice patterns, explain them clearly, and suggest how urgent something might be, so staff can react faster and with better context.

2. Why This Approach

A traditional patient monitor beeps when a number crosses a limit, but it can't explain *why* that matters, or connect multiple signals together into one coherent picture. On the other hand, letting an AI model make medical decisions on its own would be risky and hard to trust or audit.

So this project splits the work into two clearly separate jobs:

A rule-based detection layer decides *what counts as a flagged event* — using fixed, explainable thresholds and patterns. This part never guesses; it only reports what a defined rule caught.

An AI reasoning layer (built with Ollama and LangChain) takes those flagged events and turns them into a clear, human-readable explanation, adding context like the patient's history — but it never decides on its own that something is or isn't a problem.

This keeps the system safe, explainable, and still genuinely useful, since the AI's value is in connecting dots and communicating clearly, not in making the medical call.

3. The Technology Stack

Ollama — runs the AI language model locally on your own machine, so patient data never has to leave your system or go to an external service.

LangChain — the toolkit used to connect the flagged events, the patient context, and the AI model together in an organized way.

Python — used for the data simulation and rule-based detection logic.

MIMIC / PhysioNet (planned) — a real, publicly available hospital dataset that will eventually replace the synthetic data once the pipeline is proven to work.

4. The Overall Pipeline

Data layer — a stream of patient vital signs, arriving over time (currently simulated; real hospital data later).

Detection layer — watches that stream and flags anything noteworthy: a value crossing a dangerous limit, a value drifting in a bad direction over time, or multiple signals moving together in a way that confirms something real is happening.

Reasoning layer (*not built yet*) — takes a flagged event, adds relevant patient context, and produces a plain-language explanation plus a suggested urgency level.

Output layer (*not built yet*) — a simple way to actually view these explanations (a dashboard or log), so the system is usable in practice.

5. What's Been Built So Far

Step 1 — Synthetic Patient Vitals Generator

Since we don't yet have access to a real hospital dataset, this tool creates realistic *fake* patient data to build and test the rest of the system against.

What it does:

Generates heart rate, oxygen level (SpO2), and breathing rate that wobble naturally over time, the way a real monitor's readings do, rather than sitting on a flat, unrealistic line.

Lets you deliberately insert specific problem events at exact times you choose — a slow oxygen drop, a sudden fast heart rate, a slow heart rate, or a period where the sensor stops reporting data entirely.

Because you control exactly what problems were inserted and when, you have a reliable way to check whether the next part of the system (the detector) actually catches them correctly.

Why this matters: building and testing a detection system is much easier when you know exactly what should be found, rather than guessing against unlabeled real-world data from the start.

Step 2 — Rule-Based Detection Layer

This tool reads through the generated vitals and flags anything that looks concerning, using a few different kinds of checks:

Threshold checks — did a value cross a known danger line? (e.g. oxygen level dropping below a safe percentage)

Trend checks — is a value steadily drifting in a bad direction, even if it hasn't crossed a hard limit yet?

Correlation checks — are two signals moving together in a way that confirms something real is happening? For example, oxygen dropping *while* heart rate rises is a much stronger signal of a genuine problem than either change alone — it helps rule out a simple sensor glitch.

Data-quality checks — did the sensor stop sending readings altogether? This gets flagged separately from an actual clinical problem, since it usually means a wire came loose or a device glitched, not that the patient's condition changed.

A problem we ran into and fixed: because the checks run continuously, the very first version flagged the *same* underlying problem dozens of times in a row (once for every few seconds it remained true). We added a merging step that groups nearby flags of the same type into a single "episode" — much closer to how a nurse

would actually describe it ("the patient desaturated for about two and a half minutes around 8:06") rather than a flood of near-identical alerts.

The output of this step is a short, clean list of real episodes — each with a start time, end time, severity, and a plain description — ready to be handed to the AI reasoning layer next.

6. What's Left To Build

Reasoning layer: Connect Ollama + LangChain so that each detected episode, combined with basic patient context (age, known conditions, recent history), produces a clear written explanation and a suggested urgency level.

Output layer: A simple way to view these results — could start as a basic dashboard or an organized log — so the system is something a person could actually use, not just something that prints to a terminal.

Swap in real data: Once the pipeline works end-to-end on synthetic data, replace the synthetic generator with a real dataset like MIMIC or PhysioNet.

7. Key Terms, in Plain Words

LLM (Large Language Model) — the type of AI model (like the ones behind ChatGPT or Claude) that can read text and generate human-like language. Here, it will read structured event data and write a plain-language explanation of it.

Ollama — a tool that lets you run an LLM directly on your own computer, instead of sending data to an outside company's servers. Important for healthcare data privacy.

LangChain — a toolkit that helps organize how information flows into and out of an LLM — for example, combining a detected event with patient history before handing it to the model, in a repeatable, structured way.

RAG (Retrieval-Augmented Generation) — a technique where relevant background information (like a patient's history) is automatically looked up and given to the AI model alongside the question, so its answer is grounded in real, specific facts rather than general knowledge.

Threshold rule — a fixed, simple limit (e.g. "oxygen below 92% is a warning") used to catch obvious problems reliably.

Trend detection — watching whether a value is steadily moving in a bad direction over time, so problems can be caught before they cross a hard limit.

Episode / event merging — combining many small, repeated detections of the same underlying problem into one clear summary, instead of reporting the same issue dozens of times.